

Manual Técnico

1. Introducción

Este documento describe los aspectos técnicos del proyecto de la red social desarrollado en C++ con una interfaz gráfica creada en Qt Creator. El sistema utiliza estructuras de datos no lineales para gestionar eficientemente la información de usuarios, publicaciones y comentarios.

2. Requisitos del Sistema

Lenguaje de programación: C++

IDE: Qt Creator

Sistema Operativo: Libre

Librerías adicionales:

Graphviz (para la generación de gráficos)

Qt (para la interfaz gráfica)

3. Arquitectura del Sistema

El sistema se divide en varias capas:

Capa de Datos: Implementa estructuras de datos no lineales para manejar usuarios, publicaciones y comentarios.

Capa de Lógica: Contiene los algoritmos de búsqueda, inserción y eliminación en las estructuras de datos.

Capa de Presentación (Interfaz gráfica): Diseñada en Qt Creator, se encarga de la interacción del usuario con la aplicación.

4. Estructuras de Datos Utilizadas

4.1. Árbol AVL (Gestión de Usuarios)

Descripción: Un árbol AVL es un árbol binario de búsqueda balanceado que se utiliza para almacenar los usuarios de la aplicación. El criterio de ordenamiento es el correo electrónico de los usuarios (orden alfabético).

Funciones principales:

Insertar usuario: Al agregar un nuevo usuario, se inserta en el árbol siguiendo las reglas del AVL para mantener el equilibrio.

Buscar usuario: La búsqueda se realiza en tiempo logarítmico gracias a las propiedades del árbol AVL.

4.2. Árbol Binario de Búsqueda (ABB) (Gestión de Publicaciones)

Descripción: Cada usuario tiene un ABB que organiza sus publicaciones por fecha. Las publicaciones más recientes están en las hojas inferiores del árbol.

Funciones principales:

Insertar publicación: Se inserta en el árbol ordenado por fecha.

Buscar publicación: Filtra las publicaciones por fecha para obtener todas las de un día específico.

4.3. Árbol B de Orden 5 (Gestión de Comentarios)

Descripción: Cada publicación puede tener comentarios, los cuales se almacenan en un árbol B de orden 5. Esto permite gestionar eficientemente grandes cantidades de comentarios.

Funciones principales:

Insertar comentario: Inserta un comentario ordenado por la fecha y hora de creación.

Buscar comentarios: Obtiene todos los comentarios de una publicación, organizados cronológicamente.

5. Implementación de Funcionalidades

5.1. Gestión de Usuarios

Descripción técnica: Los usuarios se almacenan en un árbol AVL, donde las inserciones y búsquedas se realizan en tiempo logarítmico. Los métodos de inserción, eliminación y búsqueda se implementan en clases dedicadas, cumpliendo con los requisitos de balanceo de un árbol AVL.

5.2. Publicaciones

Descripción técnica: Las publicaciones están organizadas en un ABB, donde cada nodo contiene la lista de publicaciones de una fecha específica. Las inserciones y búsquedas siguen las reglas de un árbol binario de búsqueda.

5.3. Comentarios

Descripción técnica: Los comentarios en cada publicación se gestionan con un árbol B de orden 5, lo que permite manejar de manera eficiente las inserciones y búsquedas de comentarios.

6. Uso de Graphviz

Graphviz se utiliza para la generación de gráficos que representan las estructuras de datos en la interfaz gráfica. Se generan visualizaciones del árbol AVL de usuarios, el ABB de publicaciones y el árbol B de comentarios.

7. Interfaz Gráfica (Qt Creator)

7.1. Descripción

La interfaz gráfica fue desarrollada en Qt Creator y proporciona una forma intuitiva de interactuar con las estructuras de datos. Los reportes, así como las visualizaciones de las estructuras de datos, se muestran en ventanas gráficas generadas dinámicamente.

7.2. Componentes principales

Ventana de inicio de sesión: Permite a los usuarios autenticarse en el sistema.

Panel de usuario: Muestra las publicaciones, comentarios y opciones para realizar nuevas interacciones.

Panel de administrador: Permite la gestión avanzada de usuarios y la generación de reportes.

8. Algoritmos Implementados

8.1. Inserción en Árbol AVL

cpp

Copiar código

```
void AVLTree::insert(Node* &root, string email) {  
    // Inserta un nuevo usuario y balancea el árbol si es necesario  
}
```

8.2. Inserción en Árbol Binario de Búsqueda

cpp

Copiar código

```
void ABB::insert(Date fecha, Publication pub) {  
    // Inserta una nueva publicación en el ABB, ordenada por fecha  
}
```

8.3. Inserción en Árbol B de Comentarios

cpp

Copiar código

```
void BTree::insert(Comment comment) {  
    // Inserta un comentario en el árbol B, manteniendo el orden por fecha y hora  
}
```

9. Pruebas y Validación

Cada funcionalidad fue probada individualmente mediante la inserción y búsqueda de usuarios, publicaciones y comentarios. Además, se verificó la generación correcta de reportes y gráficos utilizando Graphviz.

10. Consideraciones Finales

Este sistema fue diseñado para simular una red social básica, empleando estructuras de datos avanzadas para mejorar el rendimiento en la gestión de grandes volúmenes

de información. El uso de Qt Creator para la interfaz gráfica y Graphviz para la visualización proporciona una experiencia de usuario completa y optimizada.

Avl

```
#ifndef AVLTREE_H
#define AVLTREE_H
```

```
#include <iostream>
#include <fstream>
#include <cstdlib> // Para usar system()
#include <QString>
#include <QTableWidget>
#include "abb.h"
```

```
class Usuario {
public:
    QString nombres;
    QString apellidos;
    QString fechaNacimiento;
    QString correo;
    QString contrasena;

    Usuario(QString nombres_, QString apellidos_, QString fechaNacimiento_, QString
correo_, QString contrasena_)
        : nombres(nombres_), apellidos(apellidos_), fechaNacimiento(fechaNacimiento_),
correo(correo_), contrasena(contrasena_) {}
};
```

```
class Node {
public:
    Usuario* user; // Cambiado de int val a Usuario* user
```

```
Node* izq;  
Node* der;  
int alt;  
ABB* abbcadausr;
```

```
Node(Usuario* user_) : user(user_), izq(nullptr), der(nullptr), alt(0), abbcadausr(new  
ABB()) {}  
};
```

```
class AVLTree {
```

```
public:
```

```
Node* raiz;
```

```
AVLTree();
```

```
void add(Usuario* user); // Cambiado de void add(int val) a void add(Usuario* user);
```

```
void preorden(Node* tmp);
```

```
void enorden(Node* tmp);
```

```
void postorden(Node* tmp);
```

```
void generateDot(const std::string& filename);
```

```
// Nueva función de búsqueda por correo
```

```
Usuario* buscarPorCorreo(const QString& correo); // Busca un usuario por correo  
electrónico
```

```
Usuario* preordenLogin(Node* tmp, QString& correo, QString& contra);
```

```
Usuario* preordenBuscarCorreo(Node* tmp, const QString& correo);
```

```
// Usuario* preordenLoginSearch(Node* tmp, QString& correo);
```

```
// Usuario* preordenCase(Node* tmp);
```

```
void preordenCase(Node* tmp, QWidget* tableWidget, int& localIndex);
```

```
void postordenCase(Node* tmp, QWidget* tableWidget, int& localIndex);
```

```
void inordenCase(Node* tmp, QWidget* tableWidget, int& localIndex);
```

```
Node* preordenBuscarCorreoNodo(Node* tmp, const QString& correo);
```

```
private:
```

```

void add(Usuario* user, Node*& tmp);
int altura(Node* tmp);
Node* srl(Node* t1);
Node* srr(Node* t1);
Node* drr(Node* tmp);
Node* drr(Node* tmp);
Node* drr(Node* tmp);
int maxi(int val1, int val2);
void toDot(Node* tmp, std::ofstream& file);

// Función privada recursiva para la búsqueda
Usuario* buscarPorCorreo(Node* tmp, const QString& correo);
};

#endif // AVLTREE_H

```